TRANSACTION CONCEPT

The concept of transaction provides a mechanism for describing logical units of database processing. **Transaction processing systems** are systems with large databases and hundreds of concurrent users executing database transactions. Examples of such systems include airline reservations, banking, credit card processing, online retail purchasing, stock markets, supermarket checkouts, and many other applications. These systems require high availability and fast response time for hundreds of concurrent users. We define the concept of a transaction, which is used to represent a logical unit of database processing that must be completed in its entirety to ensure correctness.

A **transaction** is a unit of program execution that accesses and possibly updates various data items. A transaction includes one or more database access operations - these can include insertion, deletion, modification, or retrieval operations. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language (typically SQL), or programming language (for example, C++, or Java), with embedded database accesses in JDBC or ODBC. A transaction is delimited by statements (or function calls) of the form **begin transaction** and **end transaction**. The transaction consists of all operations executed between the **begin transaction** and **end transaction**. If the database operations in a transaction do not update the database but only retrieve data, the transaction is called a **read-only transaction**; otherwise it is known as a **read-write transaction**.

To ensure the integrity of data during a transaction, the database system should possess several properties, often called the **ACID properties**; the acronym is derived from the first letter of each of the four properties. They should be enforced by the concurrency control and recovery methods of the DBMS. The following are the ACID properties:

- **Atomicity:** A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.
- **Isolation:** A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary.
- **Durability or permanency:** Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

The **atomicity property** requires that we execute a transaction to completion. It is the responsibility of the transaction recovery subsystem of a DBMS to ensure atomicity. If a transaction fails to complete for some reason, such as a system crash in the midst of transaction execution, the recovery technique must undo any effects of the transaction on the database. On the other hand, write operations of a committed transaction must be eventually written to disk.

The preservation of **consistency** is generally considered to be the responsibility of the programmers who write the database programs or of the DBMS module that enforces integrity constraints. A **database state** is a collection of all the stored data items (values) in the database at a given point in time. A **consistent state** of the database satisfies the constraints specified in the schema as well as any other constraints on the database that should hold. A database program should be written in a way that guarantees that, if the database is in a consistent state before executing the transaction, it will be in a consistent state after the complete execution of the transaction, assuming that no interference with other transactions occurs.

The **isolation property** is enforced by the concurrency control subsystem of the DBMS. If every transaction does not make its updates (write operations) visible to other transactions until it is committed, one form of isolation is enforced that solves the temporary update problem and eliminates cascading rollbacks but does not eliminate all other problems. There have been attempts to define the level of isolation of a transaction. A transaction is said to have level 0 (zero) isolation if it does not overwrite the dirty reads of higher-level transactions. Level 1 (one) isolation has no lost updates, and level 2 isolation has no lost updates and no dirty reads. Finally, level 3 isolation (also called true isolation) has, in addition to level 2 properties, repeatable reads.

And last, the **durability property** is the responsibility of the recovery subsystem of the DBMS.

A SIMPLE TRANSACTION MODEL

A **data item (object)** can be a database record, but it can also be a larger unit such as a whole disk block, or even a smaller unit such as an individual field (attribute) value of some record in the database. The transaction processing concepts we discuss are independent of the data item granularity (size) and apply to data items in general. Each data item has a unique name, but this name is not typically used by the programmer; rather, it is just a means to uniquely identify each data item. For example, if the data item granularity is one disk block, then the disk block address can be used as the data item name.

To illustrate the transaction concept using a simple bank application consisting of several accounts and a set of transaction that access and update those accounts. The basic database access operations that a transaction can include are as follows:

- **read(X):** Reads a database item named X into a program variable also named X.
- **write(X):** Writes the value of program variable X into the database item named X.

Executing read(X) command includes the following steps:

1. Fin the address of the disk block that contains item X.
2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
3. Copy item X from the buffer to the program variable named X.

Executing write(X) command includes the following steps:

1. Find the address of the disk block that contains item X.
2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
3. Copy item X from the program variable named X into its correct location in the buffer.
4. Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Step 4 is the one that actually updates the database on disk. In some cases the buffer is not immediately stored to disk, in case additional changes are to be made to the buffer. Usually, the decision about when to store a modified disk block whose contents are in a main memory buffer is handled by the recovery manager of the DBMS in cooperation with the underlying operating system.

Let T_1 be transaction that transfer Rs. 100 from account A to account B:

T_1
read(A);
A := A - 100;
write(A);
read(B);
B := B + 100;
write(B);

Figure 9.1: Transaction T_1; Fund transfer of Rs. 100 from account A to account B.

Let us now consider each of the ACID properties.

- **Atomicity:** Suppose that, just before the execution of transaction T_1, account A and B each has balance of Rs. 1000. Account A is transferring Rs. 100 to Account B. This is a transaction that has two operations a) Debiting Rs. 100 from A's balance b) Creating Rs. 100 to B's balance. Let's say first operation passed successfully while second failed, in this case A's balance would be Rs. 900 while B would be having Rs. 1000 instead of Rs. 1100. This is unacceptable in a banking system. Either the transaction should fail without executing any of the operation or it should process both the operations. The Atomicity property ensures that.
- **Consistency:** The consistency requirement here is that the sum of A and B be unchanged by the execution of the transaction. Suppose, account A and B both are having a balance of Rs. 1000 and before transfer A + B = Rs. 2000. After the completion of transfer of Rs. 100 from account A to account B; Account A has Rs. 900 and Account B has Rs. 1100 which again yields A + B = Rs. 2000.
- **Isolation:** Even if the consistency and atomicity properties are ensured for each transaction, if several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state. For every pair of transactions, one transaction should start execution only when the other finished execution. In the example of consistency property we have seen A+B calculated before transaction and after the completion of transaction is preserved, if there is another concurrent transaction which reads A, B and print A+B, after the write A and read B instruction of previous transaction then value of A+B won't be preserved, resulting in inconsistent state. If the same concurrent transaction would have been executed after the completion of previous transaction then value of A+B would have preserved, resulting consistent state.
- **Durability:** Once a transaction completes successfully, the changes it has made into the database should be permanent even if there is a system failure (power failure, system crash, system error, disk failure etc.). The recovery-management component of database systems ensures the durability of transaction. Durability can be guaranteed by ensuring that either:
  - The updates carried out by the transaction have been written to disk before the transaction completes.
  - Information about the updates carried out by the transaction and written to disk is sufficient to enable the database to reconstruct the updates when the database system is restarted after the failure.

TRANSACTION ATOMICITY AND DURABILITY

A transaction is an atomic unit of work that should either be completed in its entirety or not done at all. For recovery purposes, the system needs to keep track of when each transaction starts, terminates, and commits or aborts. Therefore, the recovery manager of the DBMS needs to keep track of the following operations:

- **Begin Transaction:** This marks the beginning of transaction execution.
- **Read or Write:** These specify read or write operations on the database items that are executed as part of a transaction.
- **End Transaction:** This specifies that Read and Write transaction operations have ended and marks the end of transaction execution. However, at this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database (committed) or whether the transaction has to be aborted because it violates concurrency control or for some other reason.
- **Commit:** This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
- **Abort (or Rollback):** This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

Recovery techniques use the following operators:

- **Undo:** Similar to rollback except that it applies to a single operation rather than to a whole transaction.
- **Redo:** This specifies that certain transaction operations must be redone to ensure that all the operations of a committed transaction have been applied successfully to the database.

A transaction must be in one of the following states:

- **Active,** the initial state; the transaction stays in this state while it is executing.
- **Partially committed,** after the final statement has been executed.
- **Failed,** after the discovery that normal execution can no longer proceed.
- **Committed,** after successful completion.
- **Terminated,** releases all the resources.

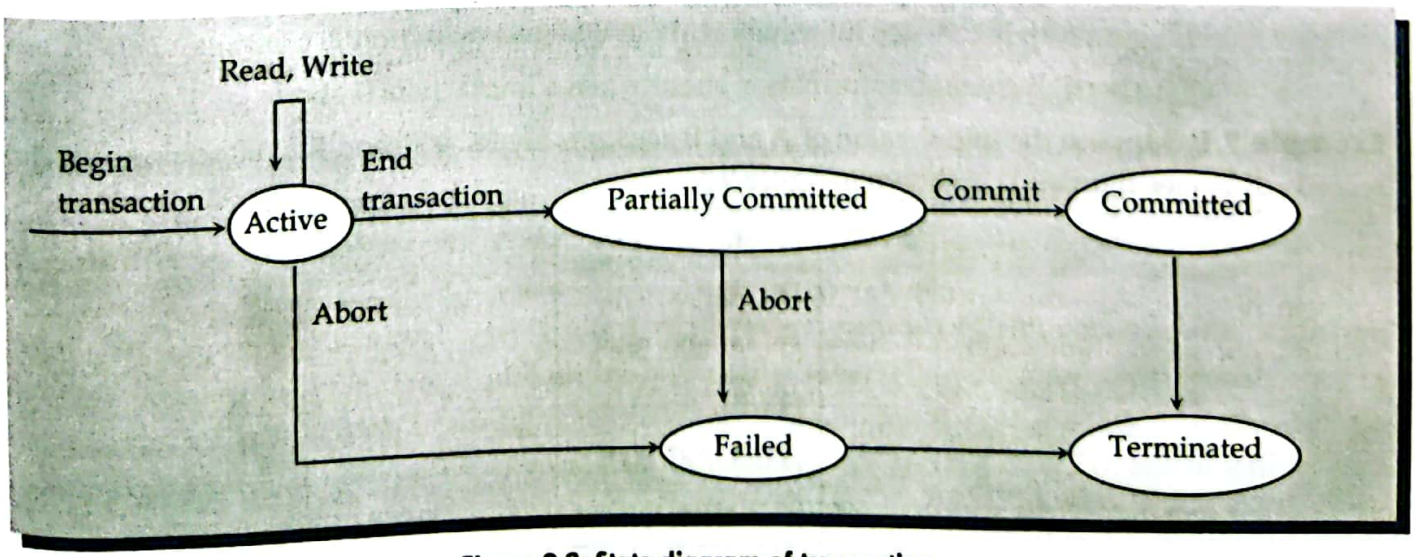


Figure 9.2: State diagram of transaction

Figure 9.2 shows a state transition diagram that illustrates how a transaction moves through its execution states. A transaction goes into an active state immediately after it starts execution, where it can execute its Read and Write operations. When the transaction ends, it moves to the partially committed state. At this point, some recovery protocols need to ensure that a system failure will not result in an inability to record the changes of the transaction permanently (usually by recording changes in the system log). Once this check is successful, the transaction is said to have reached its commit point and enters the committed state. When a transaction is committed, it has concluded its execution successfully and all its changes must be recorded permanently in the database, even if a system failure occurs. However, a transaction can go to the failed state if one of the checks fails or if the transaction is aborted during its active state. The transaction may then have to be rolled back to undo the effect of its Write operations on the database. The terminated state corresponds to the transaction leaving the system. The transaction information that is maintained in system tables while the transaction has been running is removed when the transaction terminates. Failed or aborted transactions may be restarted later—either automatically or after being resubmitted by the user—as brand new transactions.

System Log

The log, sometimes called **trail** or **journal**, is a sequence of **log records**, recording all the update activities on the database. In a stable storage, logs for each transaction are maintained. Any operation which is performed on the database is recorded is on the log. Prior to performing any modification to database, an update log record is created to reflect that modification.

An update log record represented as: $[T_i, X_j, V_1, V_2]$ has these fields:

- **Transaction identifier:** Unique Identifier of the transaction that performed the write operation.
- **Data item:** Unique identifier of the data item written.
- **Old value:** Value of data item prior to write.
- **New value:** Value of data item after write operation.

Other types of log records are:

- **[T_i start]:** It contains information about when a transaction Ti starts.
- **[T_i commit]:** It contains information about when a transaction Ti commits.
- **[T_i abort]:** It contains information about when a transaction Ti aborts.

Example 9.1: Suppose the initial value of A and B accounts are Rs. 400, and Rs. 700 respectively, log records of T_1 (see figure 9.1) is shown in figure 9.3.

[T_3 start]
[T_3, A]
[T_3, A, 400, 300]
[T_3, B]
[T_3, B, 700, 800]
[T_3 commit]

Figure 9.3: Log records for transaction T_1

Types of Failures

Failures are generally classified as transaction, system, and media failures. There are several possible reasons for a transaction to fail in the middle of execution:

1. **A computer failure (system crash):** A hardware or software error occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.
2. **A transaction or system error:** Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, the user may interrupt the transaction during its execution.
3. **Local errors or exception conditions detected by the transaction:** Certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal from that account, to be canceled. A programmed abort in the transaction causes it to fail.
4. **Concurrency control enforcement:** The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock.
5. **Disk failure:** Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.
6. **Physical problems and catastrophes:** This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

Failures of types 1, 2, 3, and 4 are more common than those of types 5 or 6. Whenever a failure of type 1 through 4 occurs, the system must keep sufficient information to quickly recover from the failure. Disk failure or other catastrophic failures of type 5 or 6 do not happen frequently; if they do occur, recovery is a major task. The concept of transaction is fundamental to many techniques for concurrency control and recovery from failures.

TRANSACTION ISOLATION

Transaction-processing systems usually allow multiple transactions to run concurrently. Allowing multiple transactions to update data concurrently causes several complications with consistency of the data, as we saw earlier. Ensuring consistency in spite of concurrent execution of transactions requires extra work; it is far easier to insist that transactions run **serially**—that is, one at a time, each starting only after the previous one has completed. However, there are two good reasons for allowing concurrency:

- **Increased processor and disk utilization,** leading to better transaction **throughput.** E.g. one transaction can be using the CPU while another is reading from or writing to the disk
- **Reduced average response time for transactions:** short transactions need not wait behind long ones.

A schedule is a collection of many transactions which is implemented as a unit. Depending upon how these transactions are arranged in within a schedule, a schedule can be of two types:

- **Serial:** The transactions are executed one after another, in a non-preemptive manner.
- **Concurrent:** The transactions are executed in a preemptive, time shared method.

In Serial schedule, there is no question of sharing a single data item among many transactions, because not more than a single transaction is executing at any point of time. However, a serial schedule is inefficient in the sense that the transactions suffer for having a longer waiting time and response time, as well as low amount of resource utilization.

In concurrent schedule, CPU time is shared among two or more transactions in order to run them concurrently. However, this creates the possibility that more than one transaction may need to access a single data item for read/write purpose and the database could contain inconsistent value if such accesses are not handled properly. Let us explain with the help of an example.

Example 9.2: Let us consider there are two transactions T_1 and T_2, whose instruction sets are given as following.

T_1
read(A);
A := A - 100;
write(A);
read(B);
B := B + 100;
write(B);

Figure 9.4: Transactions T_1 transfer Rs. 100 from account A to account B

T_2
read(A);
temp := A*0.2;
A := A - temp;
write(A);
read(B);
B := B + temp;
write(B);

Figure 9.5: Transactions T_2 transfers 20% of the balance from account A to account B.

If we prepare a serial schedule, then either T_1 will finish completely before T_2 can begin, or T_2 will finish completely before T_1 can begin.

T_1	T_2
read(A);	
A := A - 100;	
write(A);	
read(B);	
B := B + 100;	
write(B);	
commit;	
	read(A);
	temp := A*0.2;
	A := A - temp;
	write(A);
	read(B);
	B := B + temp;
	write(B);
	commit;

Figure 9.6: Schedule 1 – a serial schedule in which T_1 is followed by T_2.

Suppose, the current values of accounts A and B are same i.e. Rs. 1000; the final values of accounts A and B, after the execution of schedule 1 (see figure 9.6) are Rs. 720 and Rs. 1280, respectively. Thus the total amount of money in accounts A and B (sum A + B) is preserved after the execution of both transactions. Similarly, Sum A+B is preserved after the execution of schedule 2 (see figure 9.7), the final values of accounts A and B are Rs. 700 and Rs. 1300, respectively.

T_1	T_2
	read(A);
	temp := A*0.2;
	A := A - temp;
	write(A);
	read(B);
	B := B + temp;
	write(B);
	commit;
read(A);	
A := A - 100;	
write(A);	
read(B);	
B := B + 100;	
write(B);	
commit;	

Figure 9.7: Schedule 2 – a serial schedule in which T_2 is followed by T_1

However, if we want to create a concurrent schedule, then some Context Switching need to be made, so that some portion of T_1 will be executed, then some portion of T_2 will be executed and so on. For example say we have prepared the following concurrent schedule.

T_1	T_2
read(A);	
A := A - 100;	
write(A);	
	read(A);
	temp := A*0.2;
	A := A - temp;
	write(A);
read(B);	
B := B + 100;	
write(B);	
commit;	
	read(B);
	B := B + temp;
	write(B);
	commit;

Figure 9.8: Schedule 3 – a concurrent schedule equivalent to schedule 1

No problem here in schedule 3. We have made some Context Switching in this Schedule, the first one after executing the third instruction of T_1, second after executing the fourth instruction of T_2 and after executing the last statement of T_1. Here schedule 3 is equivalent to schedule. The sum A + B is indeed preserved.

It is clear that a proper Context Switching is very important in order to maintain the Consistency and Isolation properties of the transactions. But let us take another example where a wrong Context Switching can bring about disaster. Consider the following example involving the same T_1 and T_2.

T_1	T_2
read(A);	
A := A - 100;	
	read(A);
	temp := A*0.2
	A := A - temp;
	write(A);
	read(B);
write(A);	
read(B);	
B := B + 100;	
write(B);	
commit;	
	B := B + temp;
	write(B);
	commit;

Figure 9.9: Schedule 4 – a concurrent schedule resulting in an inconsistent state

Schedule 4 (see figure 9.9) is wrong, because we have made the switching at the second instruction of T_1. The result is very confusing. After the execution of this schedule, the final values of accounts A and B are Rs. 900 and Rs. 1200, respectively. The final state is an **inconsistent state**, since we have gained in Rs. 100 in the process of concurrent execution. Indeed, the sum A + B is not preserved by the execution of the two transactions.

In the above example, we detected the error simple by examining the schedule and applying common sense. But there must be some well formed rules regarding how to arrange instructions of the transactions to create error free concurrent schedules. This brings us to our next topic, the concept of Serializability.

SERIALIZABILITY

When several concurrent transactions are trying to access the same data item, the instructions within these concurrent transactions must be ordered in some way so as there are no problem in accessing and releasing the shared data item. We can ensure consistency of the database under concurrent execution by making sure that any schedule that is executed has the same effect as a schedule that could have occurred without any concurrent execution. That concurrent (non-serial) schedule which is equivalent to a serial schedules is called **serializable schedule**. For example, Schedule 3 (see figure 9.8) is serializable schedule since it is equivalent to the serial schedule 1 (see figure 9.6). There are two aspects of serializability:

- Conflict Serializability
- View Serializability

Conflict Serializability

Two instructions of two different transactions may want to access the same data item in order to perform a read/write operation. Conflict Serializability deals with detecting whether the instructions are conflicting in any way, and specifying the order in which these two instructions will be executed in case there is any conflict. A conflict arises if at least one (or both) of the instructions is a write operation. The following rules are important in Conflict Serializability:

1. If two instructions of the two concurrent transactions are both for read operation, then they are not in conflict, and can be allowed to take place in any order.
2. If one of the instructions wants to perform a read operation and the other instruction wants to perform a write operation, then they are in conflict, hence their ordering is important. If the read instruction is performed first, then it reads the old value of the data item and after the reading is over, the new value of the data item is written. It the write instruction is performed first, then updates the data item with the new value and the read instruction reads the newly updated value.
3. If both the transactions are for write operation, then they are in conflict but can be allowed to take place in any order, because the transaction do not read the value updated by each other. However, the value that persists in the data item after the schedule is over is the one written by the instruction that performed the last write.

Let us consider a schedule S in which there are two consecutive instructions, I and J, of transactions T_i and T_j, respectively ($i \neq j$). If I and J refer to different data items, then we can swap I and J without affecting the results of any instruction in the schedule. However, if I and J refer to the same data item X, then the order of the two steps may matter. Since we are dealing with only read and write instructions, there are four cases that we need to consider

1. I_i = read(X), I_j = read(X). I_i and I_j don't conflict.
2. I_i = read(X), I_j = write(X). They conflict.
3. I_i = write(X), I_j = read(X). They conflict.
4. I_i = write(X), I_j = write(X). They conflict.

It may happen that we may want to execute the same set of transaction in a different schedule on another day. Keeping in mind these rules, we may sometimes alter parts of one schedule (S) to create another schedule (S') by swapping only the non-conflicting parts of the first schedule. The conflicting parts cannot be swapped in this way because the ordering of the conflicting instructions is important and cannot be changed in any other schedule that is derived from the first. If these two schedules are made of the same set of transactions, then both S and S' would yield the same result if the conflict resolution rules are maintained while creating the new schedule. In that case the schedule S and S' would be called **Conflict Equivalent**. We say that a schedule S is conflict serializable if it is conflict equivalent to a serial schedule S'.

Example 9.3: Let's check whether a schedule is conflict serializable or not. Let's take an example of schedules.

T_1	T_2
read(A);	
write(A);	
	read(A);
	write(A);
read(B);	
write(B);	
	read(B);
	write(B);

Figure 9.10: Schedule 3 – showing only the read and write instructions

In schedule 3 shown in figure 9.10, write (A) instruction of T_1 conflicts with read (A) instruction of T_2. However, the write (A) instruction of T_2 does not conflict with the read (B) instruction of T_1 as they access different data items.

T_1	T_2
read(A);	
write(A);	
read(B);	
write(B);	
	read(A);
	write(A);
	read(B);
	write(B);

Figure 9.11: Schedule 5 – a serial schedule that is equivalent to schedule 3

Schedule 3 (see figure 9.10) can be transformed into schedule 5 (see figure 9.11), a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore schedule 3 is conflict serializable.

T_3	T_4
read(X);	
	write(X);
write(X);	

Figure 9.12: Schedule 6

Finally, consider schedule 6 of figure 9.12; it consist of only read and write operations of T_3 and T_4. This schedule is not conflict serializable, since it is not equivalent to either the serial schedule T_3 followed by T_4 or the serial schedule T_4 followed by T_3.

Testing for conflict serializability

Under the **constrained write rule** (that is, a transaction updates a data item based on its old value, which is first read by the transaction), a **Precedence** (or **Serialization) graph** can be produced to test conflict serializability of a schedule. For a schedule S, a directed Graph (V, E) consisting of a set of nodes $V = \{T_1, T_2, T_3, ..., T_n\}$ and a set of directed edges $E = \{e_1, e_2, e_3, ..., e_m\}$, which is constructed as follows:

1. Create node for each transaction
2. Create a directed edge, $T_i \rightarrow T_j$, if T_j reads a value of an item written by T_i.
3. Create a directed edge, $T_i \rightarrow T_j$, if T_j writes a value into item after it has been read by T_i.
4. Create a directed edge, $T_i \rightarrow T_j$, if T_j writes a value into an item after written by T_i.

If an edge $T_i \rightarrow T_j$ exists in the precedence graph for S, then in any serial schedule S' equivalent to S, T_i must appear before T_j. If the precedence graph contains **no cycle**, the schedule is a conflict serializable else not conflict serializable. Let's see some examples to test for conflict serializability.

Example 9.4: Consider the schedule 7 and check for conflict serializability.

T_5	T_6
read(X);	
read (Y);	
	write(X);
write(X);	
	read(Y);

Figure 9.13: Schedule 7

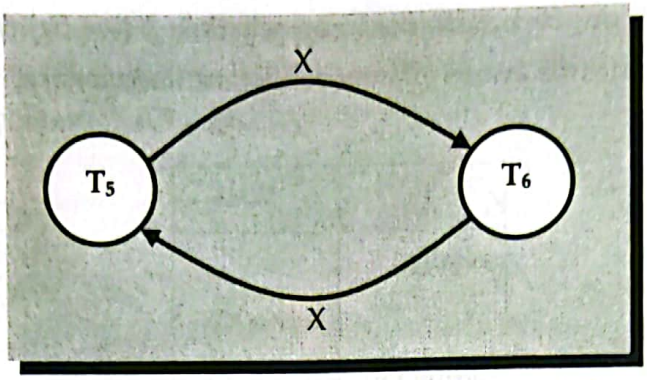


Figure 9.14: precedence graph of schedule 7

Since the graph is cyclic, we can conclude that schedule 7 is not conflict serializable.

Example 9.5: Consider the schedule 8 and check for conflict serializability.

T_7	T_8	T_9
read(A);		
		Read(A)
		write(A);
write(A);		
	read(A);	

Figure 9.15: Schedule 8

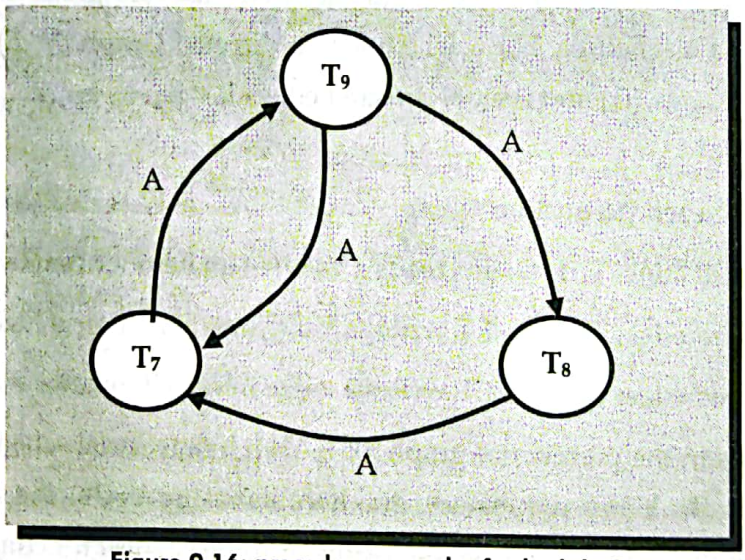


Figure 9.16: precedence graph of schedule 8

Since the graph is cyclic, we can conclude that schedule 8 is not conflict serializable.

Example 9.6: Consider the schedule 9 and check for conflict serializability.

T_{10}	T_{11}	T_{12}
read(A);		
	read(C);	
read(C);		
		Read(A)
		Read(B);
		write(A);
	read(B)	
	write(C)	
	write(B)	

Figure 9.17: Schedule 9

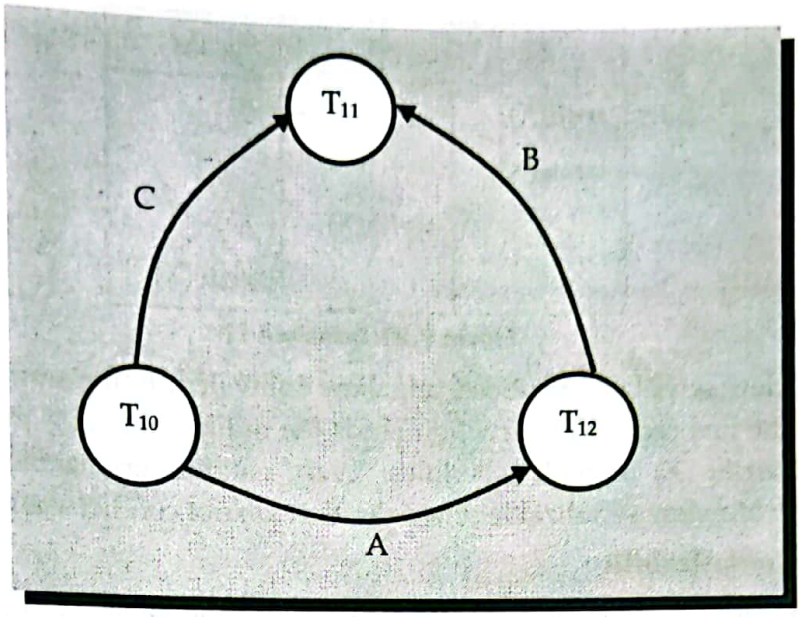


Figure 9.18: Precedence graph of schedule 9

Since the graph is acyclic, we can conclude that schedule 9 is conflict serializable.

View Serializability

There are several other types of serializability that offer less stringent definitions of schedule equivalence than that offered by conflict serializability. One less restrictive definition is called **view serializability**. Two schedules S and S′ consisting of the same operations from n transactions T_1, T_2, . . . , T_n are view equivalent if the following three conditions hold:

1. For each data item X, if transaction T_i reads the initial value of X in schedule S, then transaction T_i must also read the initial value of X in schedule S′.
2. For each read operation on data item X by transaction T_i in schedule S, if the value read by T_i has been written by transaction T_j, then transaction T_i must also read the value of X produced by transaction T_j in schedule S′.
3. For each data item X, if the last write operation on X was performed by transaction T_i in schedule S, the same transaction must perform the final write on data item X in schedule S′.

Except in these three cases any alteration can be possible while creating S′ by modifying S.

Example 9.7: Schedule 1 (see figure 9.6) is not view equivalent to schedule 2 (see figure 9.7) since in schedule 1, the value of account read by transaction T_2 was produce by T_1, where as this case does not hold in schedule 2.

Example 9.8: Consider the schedule 10 and schedule 11 as follows:

T_3	T_4	T_{13}
read(X);		
	write(X);	
write(X);		
		write(X);

Figure 9.19: Schedule 10

T_3	T_4	T_{13}
read(X);		
write(X);		
	write(X);	
		write(X);

Figure 9.20: Schedule 11

The schedule 10 is view serializable, indeed it is view equivalent to the serial schedule 11 (Schedule <T_3, T_4, T_{13}>) since the one read (X) instruction reads the initial value of X in both schedules and T_{13} performs the final write (X) in both schedules. Every conflict-serializable schedule is also view serializable but there are view serializable schedules that are not conflict serializable.

Testing for conflict serializability

The precedence graph test for conflict serializability cannot be used directly to test for view serializability. Testing for view serializability is much more complex than testing for conflict serializability. In fact, it has been shown that testing for view serializability is NP-complete; thus, it is highly improbable that an efficient algorithm can be found.

TRANSACTION ISOLATION AND ATOMICITY

So far, we have studied schedules while assuming implicitly that there are no transaction failures. We now address the effect of transaction failures during concurrent execution. If a transaction T_i fails, for whatever reason, we need to undo the effect of this transaction to ensure the atomicity property of the transaction. In a system that allows concurrent execution, the atomicity property requires that any transaction T_j that is dependent on T_i (that is, T_j has read data written by T_i) is also aborted. To achieve this, we need to place restrictions on the type of schedules permitted in the system. In the following two subsections, we address the issue of what schedules are acceptable from the viewpoint of recovery from transaction failure.

T_{14}	T_{15}
read(A);	
write(A);	
	read(A);
	commit;
read(B);	

Figure 9.21: Schedule 12 - a non-recoverable schedule

Recoverable schedule

A recoverable schedule is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the commit operation of T_j.

Example 9.19: For schedule 12 to be recoverable, T_{15} would have to delay committing until after T_{14} commits as shown in figure 9.22 (schedule 13).

T_{14}	T_{15}
read(A);	
write(A);	
	read(A);
read(B);	
commit;	
	commit;

Figure 9.22: Schedule 13 - a recoverable schedule

Cascadeless schedule

Even, if a schedule is recoverable to recover correctly from failure of transaction Ti, we may have to rollback several transactions. This phenomenon in which a single transaction failure which leads to a series of transaction rollbacks, is called **cascading rollbacks**.

Example 9.10: Schedule 14 is cascading rollbacks because if T_{16} fails T_{17} must also be rolled back.

T_{16}	T_{17}
read(A);	
write(A);	
	read(A);
	write(A);
commit;	
	commit;

Figure 9.23: Schedule 14 – cascading rollback schedule

Cascadeless schedule avoids cascading rollbacks and where for each pair of transactions T_i and T_j such that T_j reads a data item after written by T_i, then commits T_i before the read by T_j.

Example 9.11: For schedule 14 to be cascadeless schedule, T_{16} commits before read operation of T_{17} as shown in figure 9.24 (schedule 15). Schedule 15 is cascadeless schedule because if T_{16} fails T_{17} should not be rolled back.

T_{16}	T_{17}
read(A);	
write(A);	
commit;	
	read(A);
	write(A);
	commit;

Figure 9.24: Schedule 15 - cascadeless schedule

TRANSACTION ISOLATION LEVELS

Transaction isolation level is part of the isolation state of a database management system. Isolation is one of the ACID (atomicity, consistency, isolation, durability) properties which determine how transaction integrity is visible to other users and systems. It means that a transaction should take place in a system in such a way that it is the only transaction that is accessing the resources in a database system.

Isolation levels define the degree to which a transaction must be isolated from the data modifications made by any other transaction in the database system. A transaction isolation level is defined by the following phenomena:

- **Dirty Read:** A Dirty read is the situation when a transaction reads a data that has not yet been committed. For example, let's say transaction 1 updates a row and leaves it uncommitted, meanwhile, Transaction 2 reads the updated row. If transaction 1 rolls back the change, transaction 2 will have read data that is considered never to have existed.
- **Non Repeatable read**: Non Repeatable read occurs when a transaction reads same row twice, and get a different value each time. For example, suppose transaction T_1 reads data. Due to concurrency, another transaction T_2 updates the same data and commit, now if transaction T_1 rereads the same data, it will retrieve a different value.
- **Phantom Read:** Phantom Read occurs when two same queries are executed, but the rows retrieved by the two, are different. For example, suppose transaction T_1 retrieves a set of rows that satisfy some search criteria. Now, Transaction T_2 generates some new rows that match the search criteria for transaction T_1. If transaction T_1 re-executes the statement that reads the rows, it gets a different set of rows this time.

The isolation levels specified by the SQL standard are as follows:

- **Serializable** usually ensures serializable execution. However, as we shall explain shortly, some database systems implement this isolation level in a manner that may, in certain cases, allow non-serializable executions.
- **Repeatable read** allows only committed data to be read and further requires that, between two reads of a data item by a transaction, no other transaction is allowed to update it. However, the transaction may not be serializable with respect to other transactions. For instance, when it is searching for data satisfying some conditions, a transaction may find some of the data inserted by a committed transaction, but may not find other data inserted by the same transaction.
- **Read committed** allows only committed data to be read, but does not require repeatable reads. For instance, between two reads of a data item by the transaction, another transaction may have updated the data item and committed.
- **Read uncommitted** allows uncommitted data to be read. It is the lowest isolation level allowed by SQL.

CONCURRENCY CONTROL INTRODUCTION

Concurrency control is the process of managing simultaneous execution of transactions in a shared database, to ensure the serializability of transactions. Concurrent access is quite easy if all users are just reading data. There is no way they can interfere with one another. Though for any practical database, would have a mix of read and write operations, there could arise many problems if the access control is not done properly and hence the concurrency is a challenge.

Example 9.12: Consider an airline reservation database in which read is stored for each airline flight. Each record includes the number of reserved seats on that flight as a named data item among other information. A transaction T_{18} that transfer N reservations from one flight whose number of reserved seats is stored in the database item named X to another flight whose number of reserved seats is stored in the database item named Y. T_{19} represents another transaction that just reserves M seats on the first flight (X) referenced in transaction T_{18}.

(a)

T_{18}
read(X);
X:= X – N;
write(X);
read (Y);
Y:= Y + N;
write(Y);

(b)

T_{19}
read(X);
X:= X + M;
write(X);

Figure 9.25: Concurrent transactions for flight reservation seats in airline reservation database

Need for Concurrency Control

A major objective in developing a database is to enable many users to access shared data concurrently. Concurrent access is relatively easy if all users are only reading data, as there is no way that they can interfere with one another. However, when two or more users are accessing the database simultaneously and at least one is updating data, there may be interference that can result in inconsistencies. This objective is similar to the objective of multi-user computer systems, which allow two or more programs (or transactions) to execute at the same time.

However, although two transactions may be perfectly correct in themselves, the interleaving of operations in this way may produce an incorrect result, thus compromising the integrity and consistency of the database. We examine three examples of potential problems caused by concurrency: **the lost update problem, the uncommitted dependency problem,** and **the inconsistent analysis problem.** To illustrate these problems, we use a simple bank account relation using the transactions as the unit of concurrency control.

Example 9.13: The lost update problem

An apparently successfully completed update operation by one user can be overridden by another user. This is known as the **lost update problem** and is illustrated in figure 9.26, in which transaction T_{20} is executing concurrently with transaction T_{21}. T_{20} is withdrawing Rs. 10 from an account A,

initially A = Rs. 100, and T_{21} is depositing Rs. 100 into the same account. If these transactions are executed serially, one after the other with no interleaving of operations, the final balance would be Rs. 190 no matter which transaction is performed first.

Transactions T_{20} and T_{21} start at nearly the same time, and both read the balance as Rs. 100. T_{21} increases A by Rs. 100 to Rs. 200 and stores the update in the database. Meanwhile, transaction T_{20} decrements its copy of A by Rs. 10 to Rs. 90 and stores this value in the database, overwriting the previous update, and thereby "losing" the Rs. 100 previously added to the balance.

The loss of T_{21}'s update is avoided by preventing T_{20} from reading the value of A until after T_{21}'s update has been completed.

Time	T_{20}	T_{21}	A (Rs.)
t_1		read(A);	100
t_2	read(A);	A:= A + 100;	100
t_3	A:= A - 10;	write(A);	200
t_4	write(A);	commit;	90
t_5	commit;		90

Figure 9.26: Schedule 16 – the lost update problem

Example 9.14: The uncommitted dependency (or dirty read or temporary update) problem

The uncommitted dependency problem occurs when one transaction is allowed to see the intermediate results of another transaction before it has committed. Figure 9.27 (schedule 17) shows an example of an uncommitted dependency that causes an error, using the same initial value for balance A as in the previous example 9.13. Here, transaction T_{21} updates A to Rs. 200, but it aborts the transaction so that A should be restored to its original value of Rs. 100. However, by this time transaction T_{20} has read the new value of A (Rs. 200) and is using this value as the basis of the Rs. 10 reduction, giving a new incorrect balance of Rs. 190, instead of Rs. 90. The value of A, read by T_{20} is called **dirty data**, giving rise to the alternative name, the **dirty read problem**.

The reason for the rollback is unimportant; it may be that the transaction was in error, perhaps crediting the wrong account. The effect is the assumption by T_{20} that T_{21}'s update completed successfully, although the update was subsequently rolled back. This problem is avoided by preventing T_{20} from reading A until after the decision has been made to either commit or abort T_{21}'s effects.

Time	T_{20}	T_{21}	A (Rs.)
t_1		read(A);	100
t_2		A:= A + 100;	100
t_3		write(A);	200
t_4	read(A);		200
t_5	A:= A - 10;	rollback;	100
t_6	write(A);		190
t_7	commit;		190

Figure 9.27: Schedule 17 – the dirty read problem

Example 9.15: The inconsistent analysis problem

The problem of inconsistent analysis occurs when a transaction reads several values from the database but a second transaction updates some of them during the execution of the first. For example, a transaction that is summarizing data in a database (for example, totaling balances) will obtain inaccurate results if, while it is executing, other transactions are updating the database. One example is illustrated in figure 9.28 (schedule 18), in which a summary transaction T_{23} is executing concurrently with transaction T_{22}. Transaction T_{23} is totaling the balances of account A (Rs. 100), account B (Rs. 50), and account C (Rs. 25). However, in the meantime, transaction T_{22} has transferred Rs. 10 from account A to C, so that T_{23} now has the wrong result (Rs. 10 too high). This problem is avoided by preventing transaction T_{23} from reading A and C until after T_{22} has completed its updates.

Time	T_{22}	T_{23}	A (Rs.)	B (Rs.)	C (Rs.)	S (Rs.)
t_1		S = 0;	100	50	25	0
t_2	read(A);	read(A);	100	50	25	0
t_3	A:= A - 10;	S := S + A;	100	50	25	100
t_4	write(A);	read(B);	90	50	25	100
t_5	read(C);	S := S + B;	90	50	25	150
t_6	C:= C + 10;		90	50	25	150
t_7	write(C);		90	50	35	150
t_8	commit;	read(C);	90	50	35	150
t_9		S := S + C;	90	50	35	185
t_{10}		commit;	90	50	35	185

Figure 9.28: Schedule 18 – the inconsistent analysis problem

Another problem can occur when a transaction T rereads a data item it has previously read but, in between, another transaction has modified it. Thus, T receives two different values for the same data item. This is sometimes referred to as a **nonrepeatable (or fuzzy) read.** A similar problem can occur if transaction T executes a query that retrieves a set of tuples from a relation satisfying a certain predicate, re-executes the query at a later time, but finds that the retrieved set contains an additional **(phantom)** tuples that has been inserted by another transaction in the meantime. This is sometimes referred to as a **phantom read.**

Thus, Concurrency control is used to address such conflicts which mostly occur with a multi-user system. It helps user to make sure that database transactions are performed concurrently without violating the data integrity of respective databases. Therefore, concurrency control is a most important element for the proper functioning of a system where two or multiple database transactions that require access to the same data, are executed simultaneously.

Purpose of Concurrency Control

Purpose for using Concurrency control in DBMS:

- To apply Isolation through mutual exclusion between conflicting transactions
- To resolve read-write and write-write conflict issues

- To preserve database consistency through constantly preserving execution obstructions
- The system needs to control the interaction among the concurrent transactions. This control is achieved using concurrent-control schemes.
- Concurrency control helps to ensure serializability

Example 9.16: Assume that two people who go to electronic kiosks at the same time to buy a movie ticket for the same movie and the same show time. However, there is only one seat left in for the movie show in that particular theatre. Without concurrency control, it is possible that both moviegoers will end up purchasing a ticket. However, concurrency control method does not allow this to happen. Both moviegoers can still access information written in the movie seating database. But concurrency control only provides a ticket to the buyer who has completed the transaction process first.

Earlier we talked about theoretical concepts like serializability, but the practical concept of this can be implemented by using concurrency control protocols. Here we shall discuss some protocols like Lock-Based, Deadlock Handling, Multiple Granularity and Timestamp-Based to provide an environment in which concurrent transactions can preserve their Consistency and Isolation properties.

LOCK-BASED PROTOCOL

One way to ensure isolation is to require that data items be accessed in a mutually exclusive manner; that is, while one transaction is accessing a data item, no other transaction can modify that data item. Main techniques used to control concurrent execution of transactions are based on the concept of locking data items. A **lock** is a mechanism to control concurrent access to a data item. Generally, there is one lock for each data item in the database. Locks are used as a means of synchronizing the access by concurrent transactions to the database items. A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

Shared/Exclusive Locking

There are various modes in which a data item may be locked. In this section, we restrict our attention to two modes:

1. **Shared (S) Lock:** If a transaction has a shared lock on a data item, it can read the item but not update it. The lock is shared in the sense that any other transaction can acquire the shared lock on that same data item for reading purpose. **S-lock** is requested using **lock-S** instruction.
2. **Exclusive (X) Lock:** If a transaction has an exclusive lock on a data item, it can both read and update the item. The lock is exclusive in the sense that no other transaction can acquire any kind of lock (either shared or exclusive) on the same data item. **X-lock** is requested using **lock-X** instruction.

Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted. The relationship between Shared and Exclusive lock can be represented by the following table which is known as **Lock-compatibility Matrix.**

Locks to be granted \ Locks already existing	Shared (S)	Exclusive (X)
Shared (S)	TRUE	FALSE
Exclusive (X)	FALSE	FALSE

Figure 9.29: Lock-compatibility Matrix of Shared/Exclusive Locking

A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions. Any number of transactions can hold shared locks on an item, but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item. If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Example 9.17: Suppose a transaction T_{24} which transfers Rs. 100 from accounts B to A as shown in figure 9.30, performing locks the transaction should now be written as the following:

T_{24}	Description
lock-X(B);	Exclusive lock, we want to both read B's value and modify it
read(B);	
B := B - 100;	
write(B);	
unlock(B);	Unlocking B after the modification is done
lock-X(A);	Exclusive lock, we want to both read A's value and modify it
read(A);	
A := A + 100;	
write(A);	
unlock(A);	Unlocking A after the modification is done

Figure 9.30: Transactions T_{24} – performing locks

Example 9.18: Suppose a transaction T_{25} which displays the total amount of money in account A and account B i.e. sum A + B, performing lock the transaction should now be written as the following:

T_{25}	Description
lock-S(A);	Shared lock, we want to read A's value
read(A);	
unlock(A);	Unlocking A
lock-s(B);	Shared lock, we want to read B's value
read(B);	
unlock(B);	Unlocking B
display (A+B)	

Figure 9.31: Transactions T25 – performing locks

Locking as above (see figure 9.31) is not sufficient to guarantee serializability; if A and B get updated in-between the read of A and B, the displayed sum would be wrong.

Example 9.19: Suppose transactions T_{24} and T_{25} are executed concurrently, then schedule 19 (see figure 9.32) is possible and performing locks:

Suppose that the values of accounts A and B are Rs. 100 and Rs. 200, respectively. If these two transactions are executed serially, either in the order T_{24} T_{25} or the order T_{25}, T_{24}, then transaction T_{25} will display the value Rs. 300. If, however, these transactions are executed concurrently, then schedule 19, in figure 9.32 is possible. In this case, transaction T_{25} displays Rs. 200, which is incorrect. The reason for this mistake is that the transaction T_{24} unlocked data item B too early, as a result of which T_{25} saw an inconsistent state.

T_{24}	T_{25}	Concurrency-control Manager
lock-X(B);		
		grant-X(B,T_{20});
read(B);		
B := B - 100;		
write(B);		
unlock(B);		
	lock-S(A);	
		grant-S(A,T_{21})
	read(A);	
	unlock(A);	
	lock-s(B);	
		grant-S(B,T_{21})
	read(B);	
	unlock(B);	
	display (A+B)	
lock-X(A);		grant-X(A,T_{20})
read(A);		
A := A + 100;		
write(A);		
unlock(A);		

Figure 9.32: Schedule 19

The schedule 19 shows the actions executed by the transactions, as well as the points at which the concurrency-control manager grants the locks. The transaction making a lock request cannot execute its next action until the concurrency control manager grants the lock. Hence, the lock must be granted in the interval of time between the lock-request operation and the following action of the transaction.

Example 9.20: Suppose transaction T_{26} correspondence to unlocking delayed to the end of the transactions T_{24} and T_{27} correspondence to unlocking delayed to the end of the transaction T_{25} are shown in figure 9.33 and figure 9.34 respectively:

T_{26}	Description
lock-X(B);	Exclusive lock, we want to both read B's value and modify it
read(B);	
B := B - 100;	
write(B);	
lock-X(A);	Exclusive lock, we want to both read A's value and modify it
read(A);	
A := A + 100;	
write(A);	
unlock(B);	Unlocking B after the modification is done (unlocking delayed)
unlock(A);	Unlocking A after the modification is done

Figure 9.33: Transactions T_{26} – (Transaction T24 with unlocking delayed)

T_{27}	Description
lock-S(A);	Shared lock, we want to read A's value
read(A);	
lock-s(B);	Shared lock, we want to read B's value
read(B);	
display (A+B)	
unlock(A);	Unlocking A (unlocking delayed)
unlock(B);	Unlocking B (unlocking delayed)

Figure 9.34: Transactions T_{27} – (Transaction T_{25} with unlocking delayed)

T_{26}	T_{27}
lock-X(B);	
read(B);	
B := B - 100;	
write(B);	
	lock-S(A);
	read(A);
	lock-S(B);
lock-X(A);	

Figure 9.35: Partial schedule 20

Unfortunately, locking can lead to an undesirable situation. Consider the partial schedule 20 of figure 9.35 for T_{26} and T_{27}. Since T_{26} is holding an exclusive lock on B and T_{27} is requesting a shared lock on B, T_{27} is waiting for T_{26} to unlock B. Similarly, since T_{27} is holding a shared lock on A and T_{26}

is requesting an exclusive lock on A, T_{26} is waiting for T_{27} to unlock A. Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called **deadlock**. When deadlock occurs, the system must roll back one of the two transactions. Once a transaction has been rolled back, the data items that were locked by that transaction are unlocked. These data items are then available to the other transaction, which can continue with its execution. **Starvation** is also possible if concurrency control manager is badly designed. For example, a transaction may be waiting for an exclusive lock on an item, while a sequence of other transactions request and are granted a shared lock on the same item. The same transaction is repeatedly rolled back due to deadlocks. Concurrency control manager can be designed to prevent starvation.

Table 9.1: Differences between starvation Vs deadlock

Starvation	Deadlock
Starvation happens if same transaction is always chosen as victim.	A deadlock is a condition in which two or more transaction is waiting for each other.
It occurs if the waiting scheme for locked items in unfair, giving priority to some transactions over others.	A situation where two or more transactions are unable to proceed because each is waiting for one of the other to do something.
Starvation is also known as lived lock.	Deadlock is also known as circular waiting.
Avoidance: • Switch priorities so that every thread has a chance to have high priority. • Use FIFO order among competing request.	**Avoidance:** • Acquire locks are predefined order. • Acquire locks at once before starting.
It means that transaction goes in a state where transaction never progress.	It is a situation where transactions are waiting for each other.

Problems in Shard/Exclusive Locking

Problems in shared/exclusive locking are:

1. May not sufficient to produce only serializable schedule.
2. May not free from irrecoverability
3. May not free from deadlock.
4. May not free from starvation.

Lock Conversion

A transaction that holds a lock on an item A is allowed under certain condition to change the lock state from one state to another. A mechanism for upgrading a shared lock to an exclusive lock, and downgrading an exclusive lock to shared lock is called **lock conversion.**

Upgrade: Conversion from shared lock to exclusive lock is denoted by upgrade. A lock-S(A) can be upgraded to lock-X(A) if T_i is the only transaction holding the S-lock on element A. Otherwise upgrade operation will be rejected.

Downgrade: Conversion from exclusive lock to shared lock is denoted by downgrade. We may downgrade lock-X(A) to lock-S(A) when we feel that we no longer want to write on data-item A. As we were holding X-lock on A, we need not check any conditions.

Two Phase Locking Protocol

The use of locks has helped us to create neat and clean concurrent schedule. The Two Phase Locking Protocol (2PL) defines the rules of how to acquire the locks on a data item and how to release the locks to ensure the serializability. The Two Phase Locking Protocol assumes that a transaction can only be in one of two phases: an **expanding** or **growing (first) phase** and a **shrinking (second) phase.**

1. **Growing Phase:** In this phase the transaction can only acquire locks, but cannot release any lock. The transaction enters the growing phase as soon as it acquires the first lock it wants. From now on it has no option but to keep acquiring all the locks it would need. It cannot release any lock at this phase even if it has finished working with a locked data item. Ultimately the transaction reaches a point where all the lock it may need has been acquired. This point is called **Lock Point.**
2. **Shrinking Phase:** After Lock Point has been reached, the transaction enters the shrinking phase. In this phase the transaction can only release locks, but cannot acquire any new lock. The transaction enters the shrinking phase as soon as it releases the first lock after crossing the Lock Point. From now on it has no option but to keep releasing all the acquired locks.

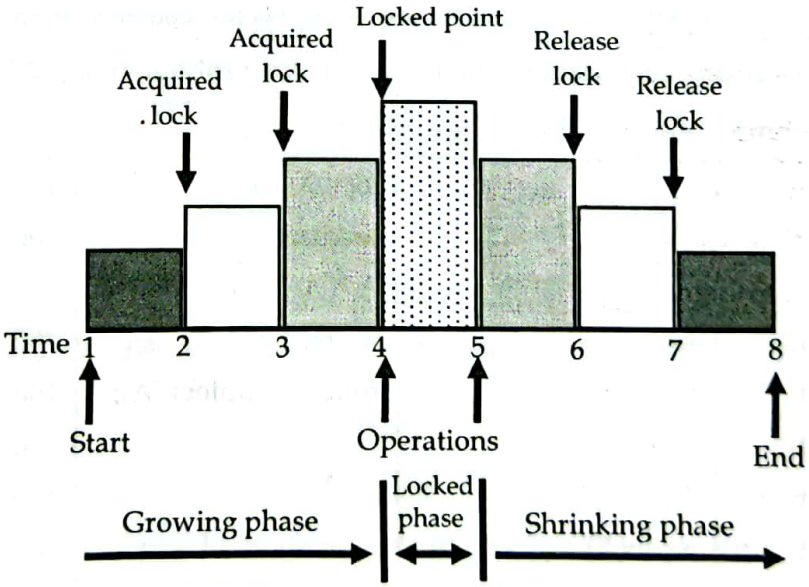


Figure 9.36: 2PL growing and shrinking phase.

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests. Thus two phase locking protocol ensures conflict serializability. If lock conversion is allowed, then upgrading of locks (from shared locked to exclusive locked) must be done during the growing phase, and downgrading of locks (from exclusive locked to shared locked) must be done in the shrinking phase. Hence, a lock-X(X) operation that upgrades an already held shared lock on X can appear only in the growing phase and a lock-S(X) operation that downgrades an already held exclusive lock on X can appear only in the shrinking phase.

Example 9.21: Preventing the lost update problem using 2PL

A solution to the lost update problem (see figure 9.26) is shown in figure 9.37 (schedule 21). To prevent the lost update problem occurring, T_{21} first requests an exclusive lock on A. It can then proceed to read the value of A from the database, increment it by Rs. 100, and write the new value back to the database. When T_{20} starts, it also requests an exclusive lock on A. However, because the data item A is currently exclusively locked by T_{21}, the request is not immediately granted and T_{20} has to wait until the lock is released by T_{21}. This occurs only once the commit of T_{21} has been completed.

Time	T_{20}	T_{21}	A (Rs.)
t_1		lock-X(A);	100
t_2	lock-X(A)	read(A);	100
t_3	wait	A:= A + 100;	100
t_4	wait	write(A);	200
t_5	wait	commit/unlock(A);	200
t_6	read(A);		200
t_7	A:= A - 10;		200
t_8	write(A);		190
t_9	commit/unlock(A);		190

Figure 9.37: Schedule 21 – preventing the lost update problem

Example 9.22: Preventing the uncommitted dependency problem using 2PL

Time	T_{20}	T_{21}	A (Rs.)
t_1		lock-X(A)	100
t_2		read(A);	100
t_3		A:= A + 100;	100
t_4	lock-X(A)	write(A);	200
t_5	wait	rollback/unlock(A);	100
t_6	read(A);		100
t_7	A:= A - 10;		100
t_8	write(A);		90
t_9	commit/unlock(A);		90

Figure 9.38: Schedule 22 – preventing the uncommitted dependency problem

A solution to the uncommitted dependency problem (see figure 9.27) is shown in figure 9.38 (schedule 22). To prevent this problem occurring, T_{21} first requests an exclusive lock on A. It can then proceed to read the value of A from the database, increment it by Rs. 100, and write the new value back to the database. When the rollback is executed, the updates of transaction T_{21} are undone and the value of A in the database is returned to its original value of Rs. 100. When T_{20} starts, it also requests an exclusive lock on A. However, because the data item A is currently exclusively locked by T_{21}, the request is not immediately granted and T_{20} must wait until the lock is released by T_{21}. This occurs only when the rollback of T_{21} has been completed.

Example 9.23: Preventing the inconsistent analysis problem using 2PL

A solution to the inconsistent analysis problem (see figure 9.28) is shown in figure 9.39 (schedule 23). To prevent this problem from occurring, T_{22} must precede its reads by exclusive locks, and T_{23} must precede its reads with shared locks. Therefore, when T_{22} starts, it requests and obtains an exclusive lock on A. Now, when T_{23} tries to share lock A, the request is not immediately granted and T_{23} has to wait until the lock is released, which is when T_{22} commits.

Time	T_{22}	T_{23}	A (Rs.)	B (Rs.)	C (Rs.)	S (Rs.)
t_1		S = 0;	100	50	25	0
t_2	lock-X(A);		100	50	25	0
t_3	read(A);	lock-S(A);	100	50	25	0
t_4	A:= A - 10;	wait	100	50	25	0
t_5	write(A);	wait	90	50	25	0
t_6	lock-X(C);	wait	90	50	25	0
t_7	read(C);	wait	90	50	25	0
t_8	C:= C + 10;	wait	90	50	25	0
t_9	write(C);	wait	90	50	35	0
t_{10}	commit/unlock(A,C);	wait	90	50	35	0
		read(A);	90	50	35	0
		S := S + A;	90	50	35	90
		lock-S(B);	90	50	35	90
		read(B);	90	50	35	90
		S := S + B;	90	50	35	140
		lock-S(C);	90	50	35	140
		read(C);	90	50	35	140
		S := S + C;	90	50	35	175
		Commit/unlock(A,B,C);	90	50	35	175

Figure 9.39: Schedule 23 – preventing the inconsistent analysis problem

Types of 2PL

Two phase locking can be divided into following types:

- **Basic 2PL:** Transaction locks data items incrementally. This may cause the problem of deadlock.
- **Conservative 2PL:** Prevents deadlock by locking all desired data items before transaction begins execution and can avoid cascading rollback. However, it is difficult to use in practice because of the need to pre-declare the read-set and write-set, which is not possible in most situations.
- **Strict 2PL:** A stricter version of basic 2PL where unlocking is performed after a transaction terminates (commits or aborts and rolled-back). Hence, no other transaction can read or write an item that is written by transaction. This is the most commonly used two-phase locking type. It can avoid cascading rollback but deadlock may occur.

- **Rigorous 2PL:** A more restrictive type of strict 2PL is rigorous 2PL, which also guarantees strict schedule. In this type, a transaction does not release any of its locks (exclusive or shared) until after it commits or aborts. In this protocol transactions can be serialized in the order in which they commit. It can avoid cascading rollback but deadlock may occur.

DEAD-LOCK HANDLING

A system is in a deadlock state if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set. Deadlock is said to be one of the most feared complications in DBMS as no task ever gets finished and is in waiting state forever.

Time	T_{28}	T_{29}
t_1	lock-X(A)	lock-X(B);
t_2	read(A);	read(B);
t_3	A := A - 10;	B := B + 100;
t_4	write(A);	write(B);
t_5	lock-X(B);	lock-X(A);
t_6	wait	wait
t_7	wait	wait
t_8	.	.
t_9	.	.
t_{10}	.	.

Figure 9.40: Schedule 24 – deadlock between two transactions.

Example 9.24: Figure 9.40 shows two transactions, T_{28} and T_{29} on schedule 24, that are deadlocked because each is waiting for the other to release a lock on an item it holds. At time t_1, transaction T_{28} requests and obtains an exclusive lock on item A and at time t_1 transaction T_{29} obtains an exclusive lock on item B. Then at t_5, T_{28} requests an exclusive lock on item B. Because T_{29} holds an exclusive lock on B, transaction T_{28} waits. Meanwhile, at time t_5, T_{29} requests an exclusive lock on item A, which is held by transaction T_{28}. Neither transaction can continue, because each is waiting for a lock it cannot obtain until the other completes. Once deadlock occurs, the applications involved cannot resolve the problem. Instead, the DBMS has to recognize that deadlock exists and break the deadlock in some way.

Unfortunately, there is only one way to break deadlock: abort one or more of the transactions. This usually involves undoing all the changes made by the aborted transaction(s). In figure 9.40, we may decide to abort transaction T_{29}. Once this is complete, the locks held by transaction T_{29} are released and T_{28} is able to continue again. Deadlock should be transparent to the user, so the DBMS should automatically restart the aborted transaction(s). However, in practice the DBMS cannot restart the aborted transaction since it is unaware of the transaction logic even if it was aware of the transaction history (unless there is no user input in the transaction or the input is not a function of the database state).

There are three general techniques for handling deadlock: **timeouts, deadlock prevention,** and **deadlock detection and recovery**. With **timeouts**, the transaction that has requested a lock waits for at most a specified period of time. By using **deadlock prevention** the DBMS looks ahead to determine if a transaction would cause deadlock and never allows deadlock to occur. Using **deadlock detection and recovery**, the DBMS allows deadlock to occur but recognizes occurrences of deadlock and breaks them. Because it is more difficult to prevent deadlock than to use timeouts or test for deadlock and break it when it occurs, systems generally avoid the deadlock prevention method.

Timeouts

A simple approach to deadlock prevention is based on lock **timeouts**. In this approach, a transaction that requests a lock will wait for only a system-defined period of time. If the lock has not been granted within that time, the transaction is said to time out. In this case, the DBMS assumes that the transaction may be deadlocked, even though it may not be, and it aborts and automatically restarts the transaction. This is a very simple and practical solution to deadlock prevention that is used by several commercial DBMSs.

The **timeout scheme** is particularly easy to implement, and works well if transactions are short and if long waits are likely to be due to deadlocks. However, in general it is hard to decide how long a transaction must wait before timing out. Too long a wait results in unnecessary delays once a deadlock has occurred. Too short a wait results in transaction rollback even when there is no deadlock, leading to wasted resources. Starvation is also a possibility with this scheme. Hence, the timeout-based scheme has limited applicability.

Deadlock Prevention

Deadlock prevention protocols ensure that the system will never enter into a deadlock state. There are two approaches to deadlock prevention strategies:

1. First approach ensures that no cyclic waits can occur by ordering the requests for locks (or all locks to be acquired together) which require that each transaction locks all its data items before it begins execution (pre-declaration). Moreover, either all are locked in one step or none are locked.
2. Second approach is closer to deadlock recovery, and performs transaction rollback instead of waiting for lock, whenever the wait could potentially result in a deadlock. This approach for preventing deadlocks is to impose an ordering of all data items, and to require that a transaction lock data items only in a sequence consistent with the ordering.

Following schemes use transaction timestamps to order transactions for the sake of deadlock prevention:

- **Wait-Die Scheme:** It is a non-preemptive technique. Wait-Die, allows only an older transaction to wait for a younger one, otherwise the transaction is aborted (dies) and restarted with the same timestamp, so that eventually it will become the oldest active transaction and will not die.
- **Wound-Wait Scheme:** It is a preemptive technique. Wound-Wait, uses a symmetrical approach: only a younger transaction can wait for an older one. If an older transaction requests a lock held by a younger one, the younger one is aborted (wounded).

Both in Wait-Die and in Wound-Wait Schemes, a rolled back transaction is restarted with its original timestamp. Older transactions thus have precedence over newer ones, and starvation is hence avoided. A variant of 2PL, called **conservative 2PL**, can also be used to prevent deadlock. In conservative 2PL, a transaction obtains all its locks when it begins, or it waits until all the locks are available.

Deadlock Detection and Recovery

If a system does not employ some protocol that ensures deadlock freedom, then a detection and recovery scheme must be used. An algorithm that examines the state of the system is invoked periodically to determine whether a deadlock has occurred. If one has, then the system must attempt to recover from the deadlock. To do so, the system must:

- Maintain information about the current allocation of data items to transactions, as well as any outstanding data item requests.
- Provide an algorithm that uses this information to determine whether the system has entered a deadlock state.
- Recover from the deadlock when the detection algorithm determines that a deadlock exists.

Deadlock Detection

Deadlock detection is usually handled by the construction of a **wait-for graph (WFG)** that shows the transaction dependencies; that is, transaction T_i is dependent on T_j if transaction T_j holds the lock on a data item that T_i is waiting for. The WFG is a directed graph G = (V, E), that consists of a set of vertices (nodes) V and a set of directed edges E, which is constructed as follows:

- Create a node for each transaction.
- Create a directed edge Ti → T_j, if transaction T_i is waiting to lock a item that is currently locked by T_j.

Deadlock exists in the system if and only if the wait-for graph contains a cycle. Each transaction involved in the cycle is said to be deadlocked. To detect deadlocks, the system needs to maintain the wait-for graph, and periodically to invoke an algorithm that searches for a cycle in the graph.

Example 9.25: Figure 9.41 shows the WFG for the transactions in Figure 9.40 (schedule 24). Clearly, the graph has a cycle in it (T_{28} → T_{29} → T_{28}), so we can conclude that the system is in deadlock.

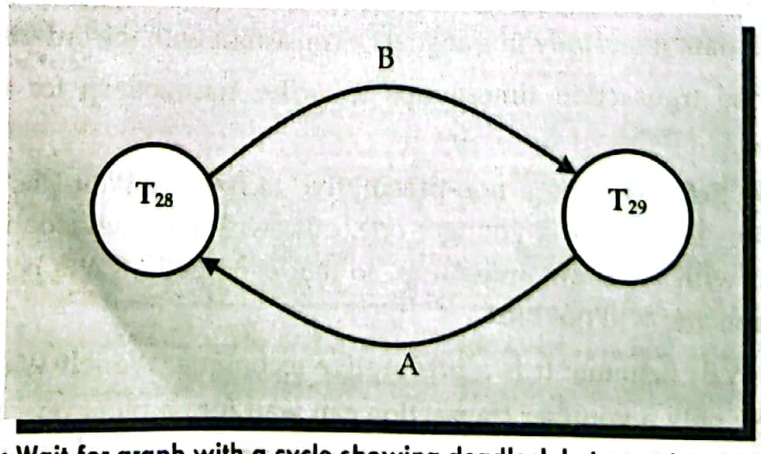


Figure 9.41: Wait-for graph with a cycle showing deadlock between two transactions.

Frequency of deadlock detection

Because a cycle in the wait-for graph is a necessary and sufficient condition for deadlock to exist, the deadlock detection algorithm generates the WFG at regular intervals and examines it for a cycle. The choice of time interval between executions of the algorithm is important. If the interval chosen is too small, deadlock detection will add considerable overhead; if the interval is too large, deadlock may not be detected for a long period. Alternatively, a dynamic deadlock detection algorithm could start with an initial interval size. Each time no deadlock is detected, the detection interval could be increased, for example, to twice the previous interval, and each time deadlock is detected, the interval could be reduced, for example, to half the previous interval, subject to some upper and lower limits.

Deadlock Recovery

When a detection algorithm determines that a deadlock exists, the system must recover from the deadlock. The most common solution is to roll back one or more transactions to break the deadlock. Three actions need to be taken:

1. **Choice of deadlock victim**: Given a set of deadlocked transactions, we must determine which transaction (or transactions) to roll back to break the deadlock. We should roll back those transactions that will incur the minimum cost. Unfortunately, the term minimum cost is not a precise one. Many factors may determine the cost of a rollback, including:
   a) How long the transaction has computed, and how much longer the transaction will compute before it completes its designated task.
   b) How many data items the transaction has used.
   c) How many more data items the transaction needs for it to complete.
   d) How many transactions will be involved in the rollback?
2. **How far to Rollback a transaction:** Once we have decided that a particular transaction must be rolled back, we must determine how far this transaction should be rolled back. Clearly, undoing all the changes made by a transaction is the simplest solution, although not necessarily the most efficient. It may be possible to resolve the deadlock by rolling back only part of the transaction.
3. **Avoiding Starvation**: Starvation occurs when the same transaction is always chosen as the victim, and the transaction can never complete. Starvation is very similar to livelock, which occurs when the concurrency control protocol never selects a particular transaction that is waiting for a lock. The DBMS can avoid starvation by storing a count of the number of times a transaction has been selected as the victim and using a different selection criterion once this count reaches some upper limit.

MULTIPLE GRANULARITY

All the concurrency control protocols that we have discussed assume that the database consists of a number of "data items," without explicitly defining the term. **Granularity** is the size of data items chosen as the **unit of protection** by a concurrency control protocol. Typically, a data item is chosen to be one of the following, ranging from coarse to fine, where **fine granularity** refers to small item sizes and **coarse granularity** refers to large item sizes:

- The entire database;
- A file;
- A page (sometimes called an area or database space – a section of physical disk in which relations are stored);
- A record;
- A field value of a record.

The size or granularity of the data item that can be locked in a single operation has a significant effect on the overall performance of the concurrency control algorithm. However, there are several tradeoffs that have to be considered in choosing the data item size. We discuss these tradeoffs in the context of locking, although similar arguments can be made for other concurrency control techniques.

Consider a transaction that updates a single tuple of a relation. The concurrency control algorithm might allow the transaction to lock only that single tuple, in which case the granule size for locking is a single record. On the other hand, it might lock the entire database, in which case the granule size is the entire database. In the second case, the granularity would prevent any other transactions from executing until the lock is released. This would clearly be undesirable. On the other hand, if a transaction updates 95% of the records in a file, then it would be more efficient to allow it to lock the entire file rather than to force it to lock each record separately. However, escalating the granularity from field or record to file may increase the likelihood of deadlock occurring.

Thus, the coarser the data item size, the lower the degree of concurrency permitted. On the other hand, the finer the item size, the more locking information that needs to be stored. The best item size depends upon the nature of the transactions. If a typical transaction accesses a small number of records, it is advantageous to have the data item granularity at the record level. On the other hand, if a transaction typically accesses many records of the same file, it may be better to have page or file granularity so that the transaction considers all those records as one (or a few) data items.

Some techniques have been proposed that have dynamic data item sizes. With these techniques, depending on the types of transaction that are currently executing, the data item size may be changed to the granularity that best suits these transactions. Ideally, the DBMS should support mixed granularity with record, page, and file-level locking. Some systems automatically upgrade locks from record or page to file if a particular transaction is locking more than a certain percentage of the records or pages in the file.

Hierarchy of granularity

We could represent the granularity of locks in a hierarchical structure where each node represents data items of different sizes, as shown in figure 9.42. Here, the root node represents the entire database, the level 1 nodes represent files, the level 2 nodes represent pages, the level 3 nodes represent records, and the level 4 leaves represent individual fields. Whenever a node is locked, all its descendants are also locked. For example, if a transaction locks a page, $Page_2$, all its records ($Record_1$ and $Record_2$) as well as all their fields ($Field_1$ and $Field_2$) are also locked. If another transaction requests an incompatible lock on the same node, the DBMS clearly knows that the lock cannot be granted.

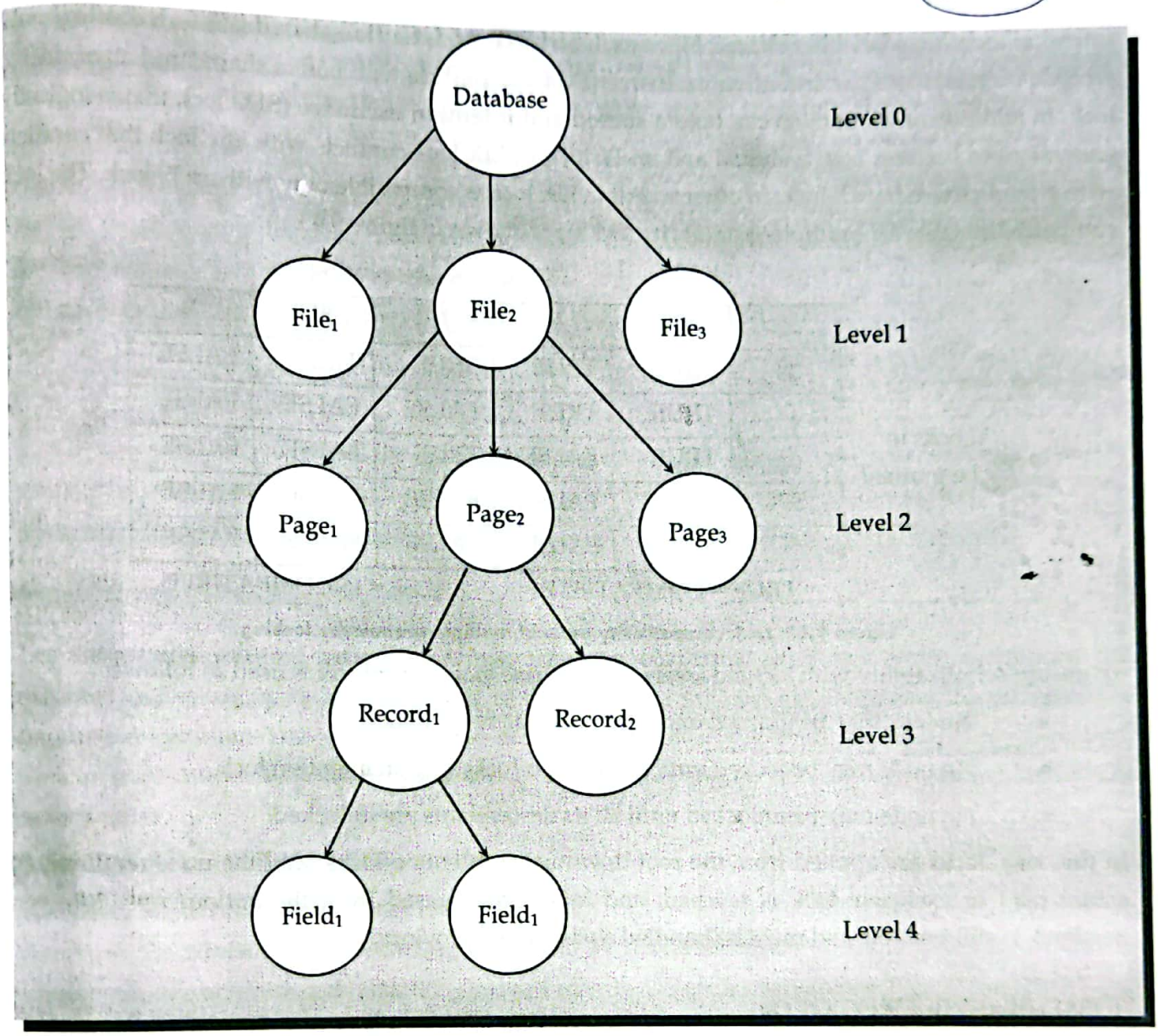


Figure 9.42: Levels of Locking

If another transaction requests a lock on any of the **descendants** of the locked node, the DBMS checks the hierarchical path from the root to the requested node to determine whether any of its ancestors are locked before deciding whether to grant the lock. Thus, if the request is for an exclusive lock on record $Record_1$, the DBMS checks its parent ($Page_2$), its grandparent ($File_2$), and the database itself to determine whether any of them are locked. When it finds that $Page_2$ is already locked, it denies the request.

Additionally, a transaction may request a lock on a node and a descendant of the node is already locked. For example, if a lock is requested on $File_2$, the DBMS checks every page in the file, every record in those pages, and every field in those records to determine whether any of them are locked.

Multiple-granularity locking

To reduce the searching involved in locating locks on descendants, the DBMS can use another specialized locking strategy called **multiple-granularity locking**. This strategy uses a new type of lock called an intention lock. When any node is locked, an intention lock is placed on all the ancestors of the node. Thus, if some descendant of $File_2$ (in our example, $Page_2$) is locked and a request is made for a lock on $File_2$, the presence of an intention lock on $File_2$ indicates that some descendant of that node is already locked.

Intention locks may be either **shared (S)** or **exclusive (X)**. An **intention shared (IS)** lock conflicts only with an exclusive lock; an **intention exclusive (IX)** lock conflicts with both a shared and an exclusive lock. In addition, a transaction can hold a **shared and intention exclusive (SIX)** lock that is logically equivalent to holding both a shared and an IX lock. A SIX lock conflicts with any lock that conflicts with either a shared or IX lock; in other words, a SIX lock is compatible only with an IS lock. The lock compatibility table for multiple-granularity locking is shown in figure 9.43.

Locks to be granted \ Locks already existing	IS	IX	S	SIX	X
IS	TRUE	TRUE	TRUE	TRUE	FALSE
IX	TRUE	TRUE	FALSE	FALSE	FALSE
S	TRUE	FALSE	TRUE	FALSE	FALSE
SIX	TRUE	FALSE	FALSE	FALSE	FALSE
X	FALSE	FALSE	FALSE	FALSE	FALSE

TRUE = COMPATIBLE FALSE = INCOMPATIBLE

Figure 9.43: Lock compatibility table of multiple-granularity locking.

To ensure serializability with locking levels, a two-phase locking protocol is used as follows:

- No lock may be granted once any node has been unlocked.
- No node may be locked until its parent is locked by an intention lock.
- No node may be unlocked until all its descendants are unlocked.

In this way, locks are applied from the root down using intention locks until the node requiring an actual read or exclusive lock is reached, and locks are released from the bottom up. However, deadlock is still possible and must be handled as discussed previously.

TIME-BASED PROTOCOL

The locking protocols that we have described thus far determine the order between every pair of conflicting transactions at execution time by the first lock that both members of the pair request that involves incompatible modes. Another method for determining the serializability order is to select an ordering among transactions in advance. The most common method for doing so is to use a **timestamp-ordering scheme / protocol**.

Timestamps

Timestamp is a unique identifier created by the DBMS that indicates the relative starting time of a transaction. Typically, timestamp values are assigned in the order in which the transactions are submitted to the system, so a timestamp can be thought of as the transaction start time. Timestamps can be generated by simply using the **system clock** at the time the transaction started, or, more normally, by incrementing a **logical counter** every time a new transaction starts.

1. Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.

2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

The timestamps of the transactions determine the serializability order. With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by $TS(T_i)$. Thus, if $TS(T_i) < TS(T_j)$, then the system must ensure that the produced schedule is equivalent to a serial schedule in which transaction T_i appears before transaction T_j.

To implement this scheme, we associate with each data item X two timestamp values:

- W_TS(X) denotes the largest timestamp of any transaction that executed write(X) successfully.
- R_TS(X) denotes the largest timestamp of any transaction that executed read(X) successfully.

These timestamps are updated whenever a new read(X) or write(X) instruction is executed.

Timestamp Ordering Protocol

The **timestamp-ordering protocol** ensures that any conflicting read and write operations are executed in timestamp order. The order of transaction is nothing but the ascending order of the transaction creation. The priority of the older transaction is higher that's why it executes first. Concurrency control techniques based on timestamp ordering do not use locks; hence, **deadlocks cannot occur.**

Basic Timestamp Ordering (TO)

Whenever some transaction T tries to issue a read(X) or a write(X) operation, the basic TO algorithm compares the timestamp of T with R_TS(X) and W_TS(X) to ensure that the timestamp order of transaction execution is not violated. If this order is violated, then transaction T is aborted and resubmitted to the system as a new transaction with a new **timestamp.**

1. Check the following condition whenever a transaction T_i issues a **read(X)** operation:
   - If $W_TS(X) > TS(T_i)$ then the operation is rejected. (Younger transaction has already written to the data item)
   - If $W_TS(X) <= TS(T_i)$ then the operation is executed.
   - Timestamps of all the data items are updated.
2. Check the following condition whenever a transaction T_i issues a **write(X)** operation:
   - If $TS(T_i) < R_TS(X)$ then the operation is rejected. (Younger Transaction has already read the data item)
   - If $TS(T_i) < W_TS(X)$ then the operation is rejected (Younger Transaction has already read the data item) and Ti is rolled back otherwise the operation is executed.

Example 9.26: To illustrate, we consider transactions T_{30} and T_{31}. Transaction T_{30} displays the contents of accounts A and B and Transaction T_{31} transfers Rs. 100 from account B to account A, and then displays the contents of both:

T_{30}	T_{31}
read(B);	
read(A);	read(B);
display(A+B);	B := B - 100;
	write(B);
	read(A);
	A := A + 100;
	write(B);
	display(A+B);

Figure 9.41: Schedule 26

In presenting schedules under the timestamp protocol, we shall assume that a transaction is assigned a timestamp immediately before its first instruction. Thus, in schedule 26 of figure 9.41, $TS(T_{30}) < TS(T_{31})$, and the schedule is possible under the timestamp protocol.

Strict Timestamp Ordering

A variation of basic TO called strict TO ensure that the schedules are both strict (for easy recoverability) and (conflict) serializable.

1. **Transaction T issues a write_item(X) operation:**
   - If TS(T) > R_TS(X), then delay T until the transaction T′ that wrote or read X has terminated (committed or aborted).
2. **Transaction T issues a read_item(X) operation:**
   - If TS(T) > W_TS(X), then delay T until the transaction T′ that wrote or read X has terminated (committed or aborted).

Thomas's Write Rule

A modification of the basic TO algorithm, known as Thomas's write rule, does not enforce conflict serializability, but it rejects fewer write operations by modifying the checks for the write(X) operation as follows:

1. If R_TS(X) > TS(T) then abort and roll-back T and reject the operation.
2. If W_TS(X) > TS(T), then just ignore the write operation and continue execution. This is because the most recent writes counts in case of two consecutive writes.
3. If the conditions given in 1 and 2 above do not occur, then execute write(X) of T and set W_TS(X) to TS(T).

15.5 Validation-Based Protocols

In cases where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low. Thus, many of these transactions, if executed without the supervision of a concurrency-control scheme, would nevertheless leave the system in a consistent state. A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alternative scheme that imposes less overhead. A difficulty in reducing the overhead is that we do not know in advance which transactions will be involved in a conflict. To gain that knowledge, we need a scheme for *monitoring* the system.

The **validation protocol** requires that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order:

1. **Read phase.** During this phase, the system executes transaction T_i. It reads the values of the various data items and stores them in variables local to T_i. It performs all write operations on temporary local variables, without updates of the actual database.
2. **Validation phase.** The validation test (described below) is applied to transaction T_i. This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.
3. **Write phase.** If the validation test succeeds for transaction T_i, the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

Each transaction must go through the phases in the order shown. However, phases of concurrently executing transactions can be interleaved.

To perform the validation test, we need to know when the various phases of transactions took place. We shall, therefore, associate three different timestamps with each transaction T_i:

1. **Start**(T_i), the time when T_i started its execution.
2. **Validation**(T_i), the time when T_i finished its read phase and started its validation phase.
3. **Finish**(T_i), the time when T_i finished its write phase.

We determine the serializability order by the timestamp-ordering technique, using the value of the timestamp Validation(T_i). Thus, the value TS(T_i) = Validation(T_i) and, if TS(T_j) < TS(T_k), then any produced schedule must be equivalent to a serial schedule in which transaction T_j appears before transaction T_k. The reason we have chosen Validation(T_i), rather than Start(T_i), as the timestamp of transaction T_i is that we can expect faster response time provided that conflict rates among transactions are indeed low.

The **validation test** for transaction T_i requires that, for all transactions T_k with TS(T_k) < TS(T_i), one of the following two conditions must hold:

1. Finish(T_k) < Start(T_i). Since T_k completes its execution before T_i started, the serializability order is indeed maintained.
2. The set of data items written by T_k does not intersect with the set of data items read by T_i, and T_k completes its write phase before T_i starts its validation phase (Start(T_i) < Finish(T_k) < Validation(T_i)). This condition ensures that the writes of T_k and T_i do not overlap. Since the writes of T_k do not affect the read of T_i, and since T_i cannot affect the read of T_k, the serializability order is indeed maintained.

T_{25}	T_{26}
read(B)	
	read(B)
	$B := B - 50$
	read(A)
	$A := A + 50$
read(A)	
< validate>	
display($A + B$)	
	< validate>
	write(B)
	write(A)

Figure 15.19 Schedule 6, a schedule produced by using validation.

As an illustration, consider again transactions T_{25} and T_{26}. Suppose that TS(T_{25}) < TS(T_{26}). Then, the validation phase succeeds in the schedule 6 in Figure 15.19. Note that the writes to the actual variables are performed only after the validation phase of T_{26}. Thus, T_{25} reads the old values of B and A, and this schedule is serializable.

The validation scheme automatically guards against cascading rollbacks, since the actual writes take place only after the transaction issuing the write has committed. However, there is a possibility of starvation of long transactions, due to a sequence of conflicting short transactions that cause repeated restarts of the long transaction. To avoid starvation, conflicting transactions must be temporarily blocked, to enable the long transaction to finish.

This validation scheme is called the optimistic concurrency-control scheme since transactions execute optimistically, assuming they will be able to finish execution and validate at the end. In contrast, locking and timestamp ordering are pessimistic in that they force a wait or a rollback whenever a conflict is detected, even though there is a chance that the schedule may be conflict serializable.

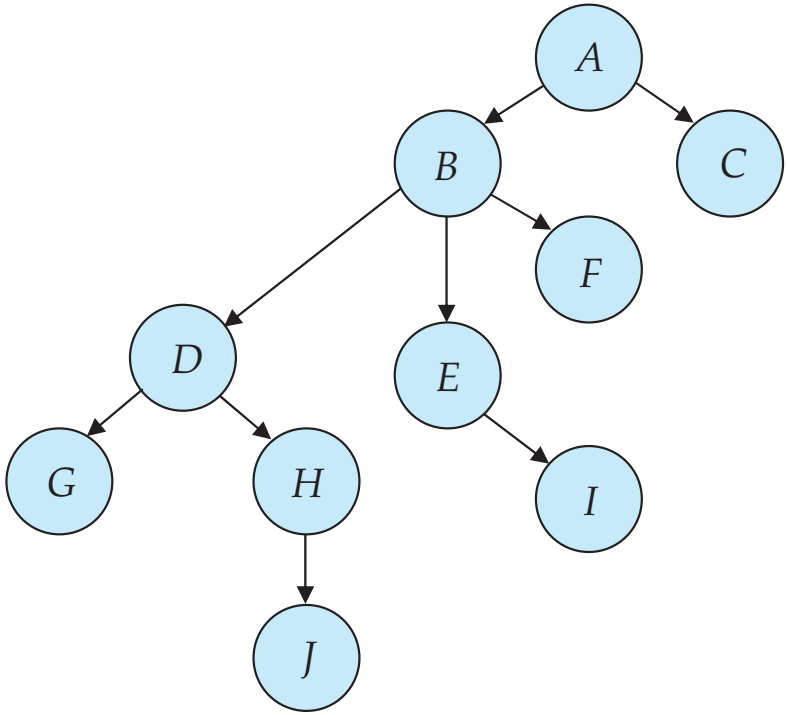


Graph-Based Protocols

- Graph-based protocols are an alternative to two-phase locking
- Impose a partial ordering $\rightarrow$ on the set **D** = $\{d_1, d_2, ..., d_h\}$ of all data items.
  - If $d_i \rightarrow d_j$ then any transaction accessing both d_i and d_j must access d_i before accessing d_j.
  - Implies that the set **D** may now be viewed as a directed acyclic graph, called a *database graph*.
- The *tree-protocol* is a simple kind of graph protocol.

©Silberschatz, Korth and Sudarshan

Tree Protocol



1. Only exclusive locks are allowed.
2. The first lock by T_i may be on any data item. Subsequently, a data Q can be locked by T_i only if the parent of Q is currently locked by T_i.
3. Data items may be unlocked at any time.
4. A data item that has been locked and unlocked by T_i cannot subsequently be relocked by T_i

©Silberschatz, Korth and Sudarshan



Graph-Based Protocols (Cont.)

- The tree protocol ensures conflict serializability as well as freedom from deadlock.
- Unlocking may occur earlier in the tree-locking protocol than in the two-phase locking protocol.
  - shorter waiting times, and increase in concurrency
  - protocol is deadlock-free, no rollbacks are required
- Drawbacks
  - Protocol does not guarantee recoverability or cascade freedom
    - Need to introduce commit dependencies to ensure recoverability
  - Transactions may have to lock data items that they do not access.
    - increased locking overhead, and additional waiting time
    - potential decrease in concurrency
- Schedules not possible under two-phase locking are possible under tree protocol, and vice versa.

©Silberschatz, Korth and Sudarshan